

Sprint Schedule Analysis

Network Dependencies & Critical Path

📅 November 16, 2025

⚠️ Executive Summary

Critical Path: F → G → H → I (10 days total)
Sprint Risk Level: Medium-High
Key Success Factor: Event Feed & Calendar path must start immediately

1 🧩 Task Dependencies

Our sprint consists of 10 tasks organized into four distinct work streams. Below is the complete breakdown of task dependencies:

1.1 🔒 Authentication Path (Independent)

- **Task C:** JWT System MVP (Login/Signup only)
→ No dependencies — Duration: 2 days
- **Task A:** Login/Signup Backend
→ Depends on Task C — Duration: 2 days
- **Task B:** Login/Signup Frontend
→ Depends on Task A — Duration: 2 days
- ⓘ *Note: This path is isolated and does not block any other features.*

1.2 🔥 Event Feed & Calendar Path (CRITICAL PATH)

- **Task F:** Event Feed Fetch (Frontend requests)
→ No dependencies — Duration: 2 days
- **Task G:** Event Feed Backend
→ Depends on Task F — Duration: 3 days
- **Task H:** Calendar View (ISO formatting)
→ Depends on Task G — Duration: 3 days
- **Task I:** Event Details Page (ISO formatting)
→ Depends on Task H — Duration: 2 days
- ⚡ **This forms the critical path: F → G → H → I (10 days total)**

1.3 Manual Event Posting Path (Independent)

- **Task D:** Post Event Backend (Manual user posting)
→ No dependencies — Duration: 2 days
- **Task E:** Post Event UI (Manual event form)
→ Depends on Task D — Duration: 2 days

1.4 Automation Path

- **Task J:** Discord Web Scraper (Automated bot)
→ Depends on Task D AND Task G — Duration: 3 days

2 Network Diagram

The network diagram below illustrates the task dependencies and highlights the critical path in red.

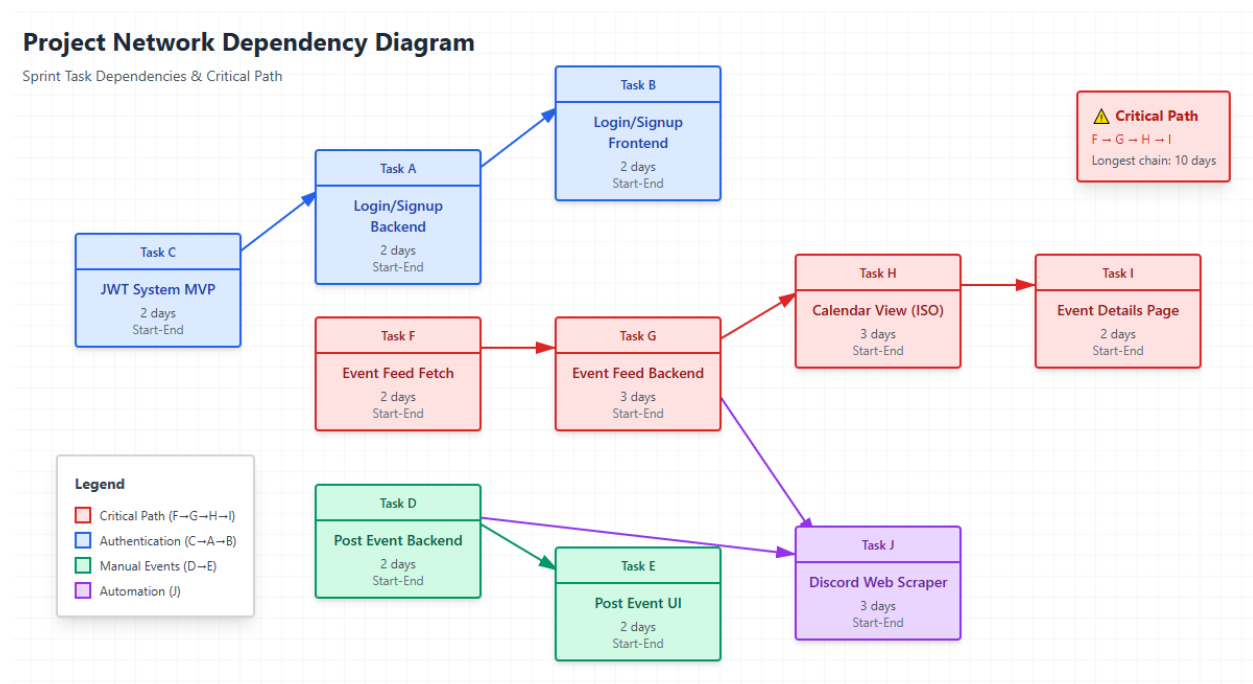


Figure 1: Project Network Dependency Diagram showing all task relationships and the critical path (F → G → H → I)

3 🧩 Critical Path Analysis

3.1 Identified Critical Path

F → G → H → I

Task F: Event Feed Fetch (2 days)

Task G: Event Feed Backend (3 days)

Task H: Calendar View (3 days)

Task I: Event Details Page (2 days)

Total: **10 days**

This represents the longest dependency chain in the project and determines the minimum time required to complete the sprint. **Any delay in these tasks will directly impact the sprint completion date.**

3.2 Why This is Critical

The event feed and calendar functionality represents the core user experience for browsing events:

1. 👤 Users cannot browse events without the event feed (Tasks F & G)
2. 📅 The calendar view depends on having a working event feed with proper data
3. ⓘ Event details pages require the calendar infrastructure
4. 📊 This forms the longest linear dependency chain in the project

4 ✅ Keeping the Sprint on Schedule

To ensure our sprint stays on schedule, we implement the following strategies:

4.1 🧠 Early Start on Critical Path Tasks

- Begin Tasks F and G on **Day 1** of the sprint
- Assign most experienced developers to critical path tasks
- Prioritize code reviews and testing for Tasks F, G, H, and I
- Hold daily standups specifically focused on critical path progress

4.2 Parallel Work Streams

- Authentication ($C \rightarrow A \rightarrow B$) can progress independently
- Manual event posting ($D \rightarrow E$) can be developed in parallel
- This allows 3 teams to work simultaneously without blocking each other
- Team members can switch to critical path tasks if needed

4.3 Risk Mitigation

-  **Buffer time:** Build 1-2 day buffer into critical path estimates
-  **Early integration:** Test F + G integration immediately after G completes
-  **API contract first:** Define event feed API schema before backend work begins
-  **Mock data:** Frontend (F) can work with mock data while backend (G) is in progress
- **ISO formatting standards:** Document date/time formatting requirements upfront to avoid rework in H and I

4.4 Progress Monitoring

- Track critical path task completion daily
- Use burn-down charts specifically for $F \rightarrow G \rightarrow H \rightarrow I$
- Identify blockers within 4 hours of occurrence
- Escalate any critical path delays to the entire team immediately

5 What Could Go Wrong: Post-Sprint Analysis

5.1 Most Likely Failure Scenarios

If we were unable to finish the sprint on time, the following scenarios (illustrated in Figure 1) would be the most probable causes:

5.1.1 Scenario 1: Event Feed Backend Delays (Task G)

What went wrong: Task G took 5 days instead of 3 days

 **Root cause:** Database schema changes, API design conflicts, or integration issues with existing systems

 **Cascade effect:** Tasks H and I are blocked, pushing sprint completion by 2 days

 **What we learned:** Should have finalized API contracts and database schemas before sprint start. Need better upfront planning for backend architecture.

5.1.2 Scenario 2: Calendar View Complexity (Task H)

What went wrong: ISO date formatting and timezone handling proved more complex than anticipated

🔍 Root cause: Underestimated complexity of calendar UI, date libraries, and timezone conversions

! Cascade effect: Task I (Event Details) is blocked, and sprint extends by 1-2 days

💡 What we learned: UI components with complex date/time requirements need prototyping before sprint. Should use established libraries (date-fns, moment.js) rather than custom solutions.

5.1.3 🏠 Scenario 3: Integration Issues Between F and G

What went wrong: Frontend fetch logic (F) and backend endpoints (G) had mismatched expectations

🔍 Root cause: Insufficient communication between frontend and backend developers, no API contract

! Cascade effect: Both tasks require rework, delaying H and I by 2-3 days

💡 What we learned: Must define and document API contracts (OpenAPI/Swagger) before development begins. Should implement contract testing to catch mismatches early.

5.2 Secondary Risk: Automation Task J

🏠 Task J Dependency Risk

While Task J (Discord Web Scraper) is not on the critical path, it has dual dependencies on Tasks D and G. If either dependency is delayed:

- Task J cannot begin until both D and G are complete
- This could push automation features to the next sprint
- **Mitigation:** Start D early since it has no dependencies; monitor G closely as it's on the critical path

6 🎓 Key Learnings & Recommendations

6.1 🏠 Architectural Insights

1. **JWT isolation is beneficial:** By keeping authentication separate from core features, we prevent auth delays from blocking the entire sprint
2. **Event feed is foundational:** Everything depends on having a working event feed. This should be the highest priority.
3. **Parallel paths enable flexibility:** Having three independent work streams (auth, events, automation) allows for resource reallocation if needed

6.2 Process Improvements

1. **Pre-sprint planning:** Define all API contracts, database schemas, and UI mockups before Day 1
2. **Critical path focus:** Assign senior developers to critical path tasks; junior developers to parallel paths
3. **Early integration:** Test integrations between F-G and G-H as soon as possible, not at the end
4. **Daily critical path reviews:** 5-minute daily standup specifically for $F \rightarrow G \rightarrow H \rightarrow I$ progress
5. **Buffer time allocation:** Add 20% buffer to critical path estimates (10 days $\rightarrow$ 12 days)

6.3 Technical Recommendations

1. Use API mocking tools (MSW, Postman Mock Server) so frontend can work independently
2. Implement contract testing (Pact) to catch integration issues early
3. Standardize on date/time libraries and ISO 8601 format from Day 1
4. Create reusable calendar components to speed up H and I
5. Document all assumptions about data formats and API responses

7 Conclusion

The critical path analysis reveals that our sprint success depends heavily on the **event feed and calendar path** ($F \rightarrow G \rightarrow H \rightarrow I$). By identifying this early, we can:

-  Allocate resources appropriately
-  Implement risk mitigation strategies
-  Enable parallel work on non-critical tasks
-  Monitor progress effectively
-  Learn from potential failures

The network diagram clearly shows that delays in the critical path will cascade, while delays in authentication or manual posting can be absorbed. This understanding allows us to make informed decisions about task prioritization and resource allocation throughout the sprint.